Real Time DevOps Daily Work — Corporate Life (A to Z Deep Production Understanding)

🟢 **1. FRESHER DEVOPS ENGINEER — 0 TO 1 YEAR**

👉 Fresher stage lo main focus **observe → understand → execute basic tasks → learn production environment**.

👉 Starting lo direct ga critical production changes ivvaru. Senior engineer instructions follow chestu system ela work avuthundo understand chestaru.

📌 **Daily Work**

👉 Jira lo assigned tickets check chestaru.

👉 Jenkins pipeline/build status check chestaru.

👉 CloudWatch / ELK / Grafana dashboards open chesi errors, CPU, memory, application health observe chestaru.

👉 Linux servers lo basic commands use chestaru.

👉 Docker containers/images basic operations chestaru.

👉 Kubernetes pods, deployments, services status check chestaru.

👉 Documentation and runbooks update chestaru.

🔍 **Ela Check Cheyyali?**

Linux server:

uptime
df -h
free -m
top
ps -ef

👉 Disk full unda?

df -h

👉 Memory usage check:

free -m

👉 Running processes:

ps -ef

☸️ **Kubernetes Basic Check**

kubectl get nodes
kubectl get pods -A
kubectl get deployments -A
kubectl get services -A

👉 Pod problem unda?

kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name>

🐬 **Docker Basic Check**

docker ps
docker images
docker logs <container-id>

🚨 **Real Example**

👉 Suppose production application slow ga undi.

👉 Senior cheptadu:

“Check whether any pod is restarting and check application logs.”

👉 First:

kubectl get pods -A

👉 CrashLoopBackOff kanipisthe:

kubectl logs <pod-name>

👉 Previous container logs kavali ante:

kubectl logs <pod-name> --previous

👉 App error identify chesi evidence senior engineer ki provide chestaru.

📌 **Fresher goal:** Immediate ga everything fix cheyyadam kaadu. **Correct information collect chesi safe ga escalate cheyyadam.**

🧠 **Fresher Mindset**

Observe → Learn → Check → Document → Escalate

🟡 **2. JUNIOR DEVOPS ENGINEER — 1 TO 3 YEARS**

👉 Junior level ki vachaka actual hands-on responsibility increase avtundi.

👉 Ippudu simple monitoring matrame kaadu; **build failures, deployments, Kubernetes issues, infrastructure problems troubleshoot** cheyyali.

⚙️ **CI/CD Pipeline Work**

👉 Developer code push chestadu.

👉 Jenkins / GitHub Actions pipeline trigger avtundi.

👉 Build fail ayithe DevOps engineer reason identify cheyyali.

🔍 **First Pipeline Check**

git status
git log --oneline -5

👉 Jenkins console logs open chesi exact failure line identify chestaru.

🚨 **Example**

Pipeline:
Checkout
↓
Build
↓
Test
↓
Docker Build
↓
Security Scan
↓
Push Image
↓
Deploy

👉 Docker build stage fail ayindi anuko.

👉 Logs lo:

ERROR: package not found

👉 App dependency/configuration issue identify chestaru.

📌 **Important:** Blind ga pipeline rerun cheyyakunda **failure reason first identify cheyyali.**

☸️ **3. KUBERNETES TROUBLESHOOTING**

👉 Junior DevOps ki Kubernetes troubleshooting chala important.

🔍 **Pod Status**

kubectl get pods -A

Common states:
Running
Pending
CrashLoopBackOff
ImagePullBackOff
ErrImagePull
Completed

🔴 **CrashLoopBackOff**

👉 Container repeatedly crash avtundi.

First:
kubectl logs <pod-name>
Then:
kubectl describe pod <pod-name>

👉 Application configuration, environment variables, secrets, dependencies check chestaru.

🔴 **ImagePullBackOff**

👉 Kubernetes Docker image pull cheyyalekapothundi.

Check:
kubectl describe pod <pod-name>
👉 Image name/tag correct aa check cheyyali.
kubectl get deployment <deployment-name> -o yaml
👉 Private registry ayithe image pull secret / IAM permissions kuda verify cheyyali.

🔴 **Pending Pod**
👉 Pod schedule avvakapothe:
kubectl describe pod <pod-name>
👉 Events section lo reason chudali.
Possible reasons:

- CPU insufficient
- Memory insufficient
- Node selector mismatch
- Taints/tolerations issue
- PVC unavailable

📌 **Real DevOps approach:**
Status → Describe → Logs → Events → Root Cause → Fix → Verify

---

🚀 **4. DEPLOYMENT & ROLLBACK**
👉 Junior engineer deployments execute cheyyadam start chestadu.
Deployment check:
kubectl get deployment
kubectl rollout status deployment/<deployment-name>
👉 New version deploy ayyaka pods healthy ga unnaya check:
kubectl get pods
👉 Problem vaste rollout history:
kubectl rollout history deployment/<deployment-name>
👉 Previous version ki rollback:
kubectl rollout undo deployment/<deployment-name>
👉 Verify:
kubectl rollout status deployment/<deployment-name>
📌 **Meaning:** Production issue vaste previous stable version ki quickly return cheyyadam.

🖥️ **5. SERVER HEALTH CHECKING**
👉 EC2/server performance kuda monitor cheyyali.
CPU
top
Memory
free -m
Disk
df -h
Disk directories
du -sh /*
Running services
systemctl status <service-name>
Logs
journalctl -u <service-name>
📌 **Example:**
👉 Disk 95% full undi.
👉 First identify:
df -h
👉 Large directory:
du -sh /var/*
👉 Logs unnecessarily grow avtunnaya check chestaru.

👉 Cleanup policy / log rotation implement chestaru.

🔵 **6. MID-LEVEL DEVOPS — AROUND 3 TO 5 YEARS**

👉 Ikkada **ownership** start avtundi.

👉 “Someone told me to fix it” level nundi:

“This system is my responsibility.”

level ki move avutaru.

📌 **Daily Responsibilities**

👉 Production deployments manage cheyyadam.

👉 Kubernetes cluster health monitor cheyyadam.

👉 CI/CD optimization.

👉 Auto-scaling issues troubleshoot cheyyadam.

👉 Production incidents handle cheyyadam.

👉 AWS infrastructure maintain cheyyadam.

👉 Cloud cost optimize cheyyadam.

👉 Developers + QA + Security teams tho coordinate cheyyadam.

📈 **7. HPA / AUTO-SCALING TROUBLESHOOTING**

👉 Traffic suddenly increase ayindi.

👉 Expected:

5 Pods

↓

High traffic

↓

HPA detects utilization

↓

10 Pods

↓

20 Pods

🔍 **Check HPA**

kubectl get hpa

Detailed:

kubectl describe hpa <hpa-name>

👉 Pods increase avtunnaya?

kubectl get pods -w

👉 Metrics available unnaya?

kubectl top pods

kubectl top nodes

📌 Issue unte check cheyyalsina layers:

Metrics → HPA → Deployment → Node capacity → Load balancer → Application

---

🚨 **8. PRODUCTION INCIDENT HANDLING**

👉 Production lo website down ayindi anuko.

👉 Mid-level engineer immediate ga panic avvakunda structured process follow chestadu.

STEP 1 — ALERT

CloudWatch / Prometheus / Grafana alert:

HTTP 5xx increased

CPU 95%

Latency high

STEP 2 — IMPACT CHECK

👉 Full application down aa?

👉 Only payment service aa?

👉 Only one region aa?

STEP 3 — CHECK APPLICATION

kubectl get pods -A

kubectl get svc -A
kubectl get ingress
STEP 4 — CHECK LOGS
kubectl logs <pod-name>
STEP 5 — CHECK RECENT DEPLOYMENT
kubectl rollout history deployment/<deployment-name>
👉 Recent deployment tarvata issue start ayithe rollback consider chestaru.
kubectl rollout undo deployment/<deployment-name>
STEP 6 — VERIFY
kubectl get pods
kubectl rollout status deployment/<deployment-name>
👉 Application endpoint test chestaru.
📌 **Goal:** First restore service, then deep root-cause analysis.

💰 **9. CLOUD COST OPTIMIZATION**
👉 Mid-level engineer ki AWS bill kuda responsibility.
🔍 **Example**
👉 EC2 instances unnecessary ga 24/7 run avtunnayi.
👉 Traffic low unna time lo resources reduce cheyyachu.
👉 Kubernetes lo:
kubectl get nodes
kubectl get hpa
👉 Unused resources identify chestaru.
Optimization Areas
👉 Right-size EC2 instances.
👉 Auto Scaling configure cheyyadam.
👉 EBS unused volumes identify cheyyadam.
👉 Old snapshots review cheyyadam.
👉 S3 lifecycle policies use cheyyadam.
👉 Kubernetes resource requests/limits correct ga configure cheyyadam.
📌 **Goal:** Performance reduce cheyyakunda unnecessary cloud spending eliminate cheyyadam.

🔴 **10. SENIOR DEVOPS ENGINEER — 5+ YEARS**
👉 Senior level lo individual commands run cheyyadam kanna **system-level decisions** important.
👉 Architecture, reliability, security, disaster recovery, scalability and team ownership senior responsibility.
📌 **Major Responsibilities**
👉 Production architecture review.
👉 High Availability design.
👉 Disaster Recovery planning.
👉 Multi-AZ / Multi-Region architecture.
👉 Security architecture.
👉 Kubernetes platform design.
👉 CI/CD architecture.
👉 Incident RCA.
👉 Cost optimization strategy.
👉 Engineering standards.
👉 Junior engineers mentoring.

📐 **11. SENIOR ARCHITECTURE THINKING**
👉 Senior engineer question:
“Application down ayithe ela recover avtundi?”
Not simply:
“Pod ela restart cheyyali?”
👉 Architecture level lo:

User
↓
Route53
↓
CloudFront
↓
WAF
↓
Load Balancer
↓
EKS
↓
Microservices
↓
RDS
👉 High availability kosam:
AZ-1
├── Nodes
└── Pods

AZ-2
├── Nodes
└── Pods

AZ-3
├── Nodes
└── Pods
👉 Oka AZ fail ayina application continue avvali.
📌 **Senior mindset:** Individual component fix kanna **failure ni prevent/reduce cheyyagalige architecture design cheyyadam.**

🔐 **12. SECURITY RESPONSIBILITY**
👉 Senior DevOps engineer security kuda architecture lo include chestadu.
IAM
User → Role → Permission
👉 Least privilege follow cheyyali.
Kubernetes
kubectl get roles -A
kubectl get rolebindings -A
👉 Secrets plain text lo store cheyyakunda proper secrets management use cheyyali.
👉 Network segmentation, security groups, private subnets, encryption and audit logging verify chestaru.

🌎 **13. DISASTER RECOVERY**
👉 Suppose complete AWS region issue ayindi anuko.
👉 Senior engineer question:
“Application ni alternate region lo ela recover chestam?”
👉 Architecture lo:
Primary Region
↓
Backups / Replication
↓
Secondary Region
↓
Failover
↓
Application Recovery
👉 RTO and RPO define chestaru.

RTO = service entha time lo recover avvali
RPO = maximum entha data loss acceptable
📌 Example:
👉 RTO = 30 minutes
👉 RPO = 5 minutes
👉 Disaster vaste system 30 minutes lopu restore avvali and maximum 5 minutes data loss acceptable.

📊 **14. MONITORING & OBSERVABILITY**
👉 DevOps engineer simply CPU monitor cheyyadam kaadu.
Three major signals:
Metrics
CPU
Memory
Latency
Request rate
Error rate
Logs
Application errors
Authentication failures
Database errors
System errors
Traces
👉 Request system lo multiple microservices cross chesinappudu complete journey trace chestayi.
📌 Example:
User Request
↓
API
↓
Auth Service
↓
Product Service
↓
Database
👉 Request ekkada slow ayindo identify cheyyachu.

🔁 **15. REAL CORPORATE CI/CD FLOW**
👉 Developer code push chestadu.
Developer
↓
GitHub
↓
CI Pipeline
↓
Build
↓
Unit Test
↓
Security Scan
↓
Docker Build
↓
Container Registry
↓
Deploy
↓
Kubernetes
↓

Health Check
↓
Production

🫧 **Pipeline fail ayithe**

👉 First failed stage identify cheyyali.
👉 Logs check cheyyali.
👉 Root cause fix cheyyali.
👉 Pipeline rerun cheyyali.
👉 Production deployment mundu required approval/testing complete cheyyali.

🌅 **16. REAL CORPORATE DAY — MORNING**

👉 Day start ayyaka first production health check.

Check:

CloudWatch
Grafana
Prometheus
Jenkins
Kubernetes
Jira

👉 Overnight incidents unnaya?
👉 Failed deployments unnaya?
👉 Critical alerts unnaya?
👉 Disk/CPU/memory abnormal ga unnaya?
👉 Previous day's incidents pending unnaya?
📌 **Goal:** New work start cheyyadaniki mundu production healthy ga unda ani confirm cheyyadam.

🌞 **17. MIDDAY WORK**

👉 Developers tho deployment discussions.
👉 Jira tickets work cheyyadam.
👉 Failed pipelines troubleshoot cheyyadam.
👉 Kubernetes issues fix cheyyadam.
👉 Infrastructure changes implement cheyyadam.
👉 QA environment issues resolve cheyyadam.
👉 Security/architecture meetings attend avvadam.

Example Jira Ticket

“Payment API pods are restarting continuously.”

👉 Engineer:

kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name>

👉 Root cause identify chestadu.
👉 Fix implement chestadu.
👉 Verify chestadu.
👉 Jira ticket update chestadu.

🌆 **18. EVENING WORK**

👉 Production deployments review chestaru.
👉 Monitoring dashboards check chestaru.
👉 Pending alerts verify chestaru.
👉 Change records / documentation update chestaru.
👉 Next shift engineer ki handover chestaru.
📌 Handover lo:

What happened?
What was fixed?
What is still pending?

Any production risk?
Any scheduled deployment?

🚨 **19. P1 INCIDENT — REAL EXAMPLE**

👉 Imagine:

“E-commerce website payment service is down.”

Detection

5xx errors ↑
Latency ↑
Payment success rate ↓

Investigation

kubectl get pods -n payment
kubectl get svc -n payment
kubectl logs <payment-pod> -n payment
kubectl describe pod <payment-pod> -n payment

👉 Recent deployment check:

kubectl rollout history deployment/payment -n payment

👉 Recent release cause ayithe controlled rollback:

kubectl rollout undo deployment/payment -n payment

👉 Status:

kubectl rollout status deployment/payment -n payment

👉 Pods:

kubectl get pods -n payment

👉 Application endpoint verify chestaru.

📌 **First priority:** Customer impact reduce cheyyadam.
📌 **Second priority:** Root cause identify cheyyadam.
📌 **Third priority:** Permanent fix implement cheyyadam.

📝 **20. RCA — ROOT CAUSE ANALYSIS**

👉 Incident fix ayyaka story complete kaadu.
👉 RCA prepare chestaru.

RCA lo:

What happened?
When did it happen?
What was the impact?
What caused the issue?
How was it detected?
What was the immediate fix?
What is the permanent fix?
How can we prevent recurrence?

Example

Incident:
Payment API unavailable

Root Cause:
Incorrect environment variable

Impact:
Payment requests failed

Immediate Fix:
Rollback

Permanent Fix:
CI validation + configuration validation

Prevention:

Automated deployment checks

📌 **SRE/DevOps maturity:** Same problem malli repeat kakunda system ni improve cheyyadam.

🤖 **21. AUTOMATION — DEVOPS CORE SKILL**

👉 Repeated manual work kanipisthe DevOps engineer question:

“Idi automate cheyyacha?”

Manual

Login server
Check disk
Remove old logs
Restart service

Automation

./health-check.sh

👉 Script automatically checks:

CPU
Memory
Disk
Service
Application

👉 Further stage lo Terraform, Ansible, Jenkins, GitHub Actions, Kubernetes automation use chestaru.

📌 **Goal:** Human effort reduce + consistency increase.

🧠 **22. DEVOPS ENGINEER LEVEL PROGRESSION**

🟢 **FRESHER**

Observe + Learn + Support

👉 Commands nerchukovali.
👉 Logs read cheyyali.
👉 Basic troubleshooting understand cheyyali.
👉 Production safety understand cheyyali.

🟡 **JUNIOR**

Execute + Debug + Fix

👉 CI/CD handle cheyyali.
👉 Kubernetes troubleshoot cheyyali.
👉 Deployments and rollback handle cheyyali.
👉 Basic automation cheyyali.

🔵 **MID-LEVEL**

Own + Optimize + Respond

👉 Production ownership.
👉 Incidents handle cheyyadam.
👉 Cloud optimization.
👉 Scaling and reliability.
👉 Team coordination.

🔴 **SENIOR**

Design + Decide + Prevent

👉 Architecture design.
👉 HA/DR.
👉 Security.
👉 Reliability.
👉 Cost strategy.
👉 RCA and permanent improvements.
👉 Team mentoring.

🔥 23. REAL DEVOPS ENGINEER THINKING PROCESS

Production lo problem vachinappudu:

ALERT
↓
IMPACT
↓
CHECK
↓
ISOLATE
↓
ROOT CAUSE
↓
FIX
↓
VERIFY
↓
MONITOR
↓
RCA
↓
PREVENT

👉 Example:

Website slow
↓
Check metrics
↓
Check load balancer
↓
Check pods
↓
Check CPU/memory
↓
Check database latency
↓
Identify bottleneck
↓
Fix
↓
Verify response time
↓
Monitor
↓
Permanent optimization

📌 **Idi real production troubleshooting mindset.**

⏳ 24. WHAT A REAL DEVOPS ENGINEER ACTUALLY DOES

👉 DevOps ante only:

❌ Jenkins run cheyyadam kaadu
❌ Docker commands matrame kaadu
❌ Kubernetes commands matrame kaadu
❌ AWS console lo resources create cheyyadam matrame kaadu

👉 Real DevOps engineer:

✅ Infrastructure build chestadu
✅ CI/CD automate chestadu
✅ Applications deploy chestadu
✅ Kubernetes manage chestadu
✅ Cloud resources operate chestadu

✅ Monitoring setup chestadu
✅ Production incidents troubleshoot chestadu
✅ Security implement chestadu
✅ Cost optimize chestadu
✅ Automation create chestadu
✅ Reliability improve chestadu

🚀 **25. FINAL CORPORATE DEVOPS STORY**

Developer
↓
GitHub
↓
CI/CD
↓
Build + Test
↓
Docker
↓
Registry
↓
Kubernetes / EC2
↓
Load Balancer
↓
Application
↓
Database
↓
Monitoring
↓
Alert
↓
DevOps Engineer
↓
Troubleshoot
↓
Fix
↓
Verify
↓
RCA
↓
Automation
↓
Better Architecture

🧠🔥 **FINAL MASTER UNDERSTANDING**

👉 **Fresher:** "Problem ekkada undo identify cheyyadam nerchukunta."
👉 **Junior:** "Problem identify chesi fix chestanu."
👉 **Mid-Level:** "Production system ni own chesi incidents and performance manage chestanu."
👉 **Senior:** "Problems repeat kakunda architecture, automation, security and reliability improve chestanu."

🔥 **ONE FINAL LINE:**

👉 **"Real DevOps engineer job is not just deploying applications; it is building, automating, monitoring, troubleshooting, securing, scaling, and continuously improving the complete production system."**